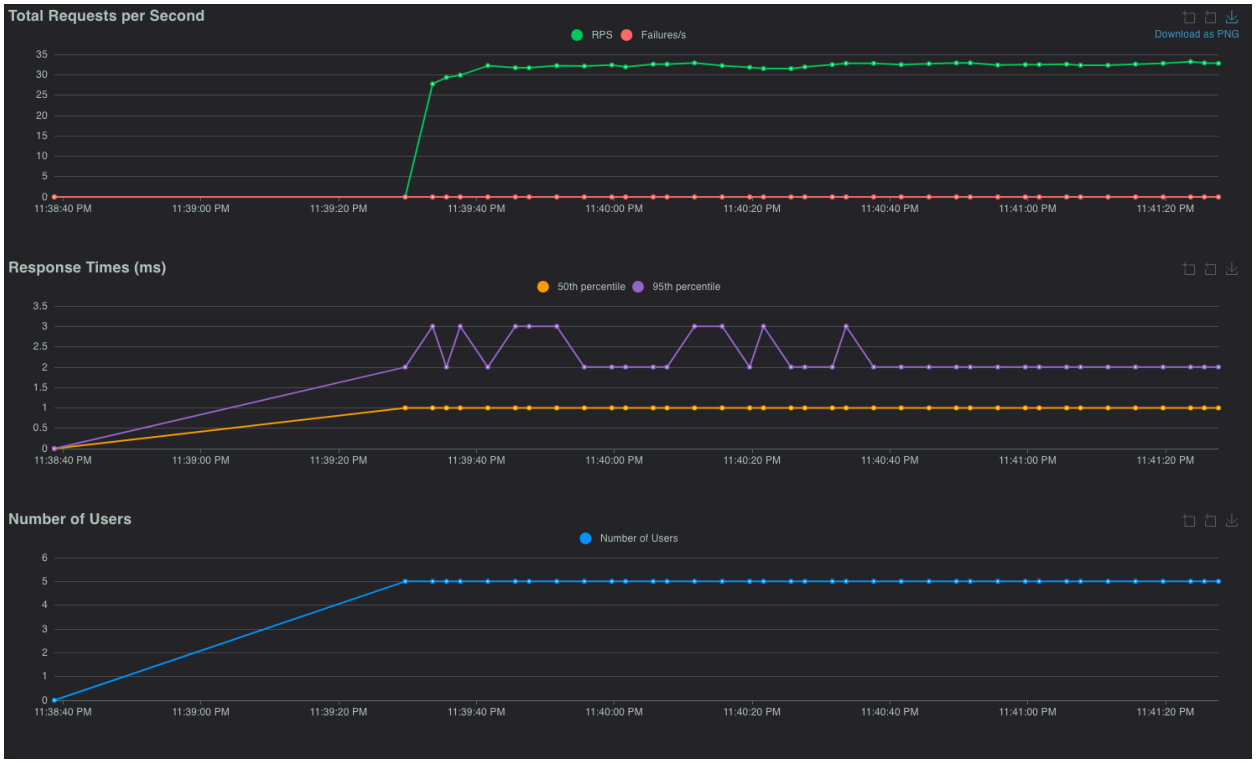
HW6-Report

Part II - Load Testing & Analysis

Local:

Test 1 - Baseline: 5 users for 2 minutes



Test 2 - Breaking Point: 20 users for 3 minutes

Despite the increased load, the service maintains consistently low response times and a 0% failure rate, indicating sufficient capacity under the tested conditions.

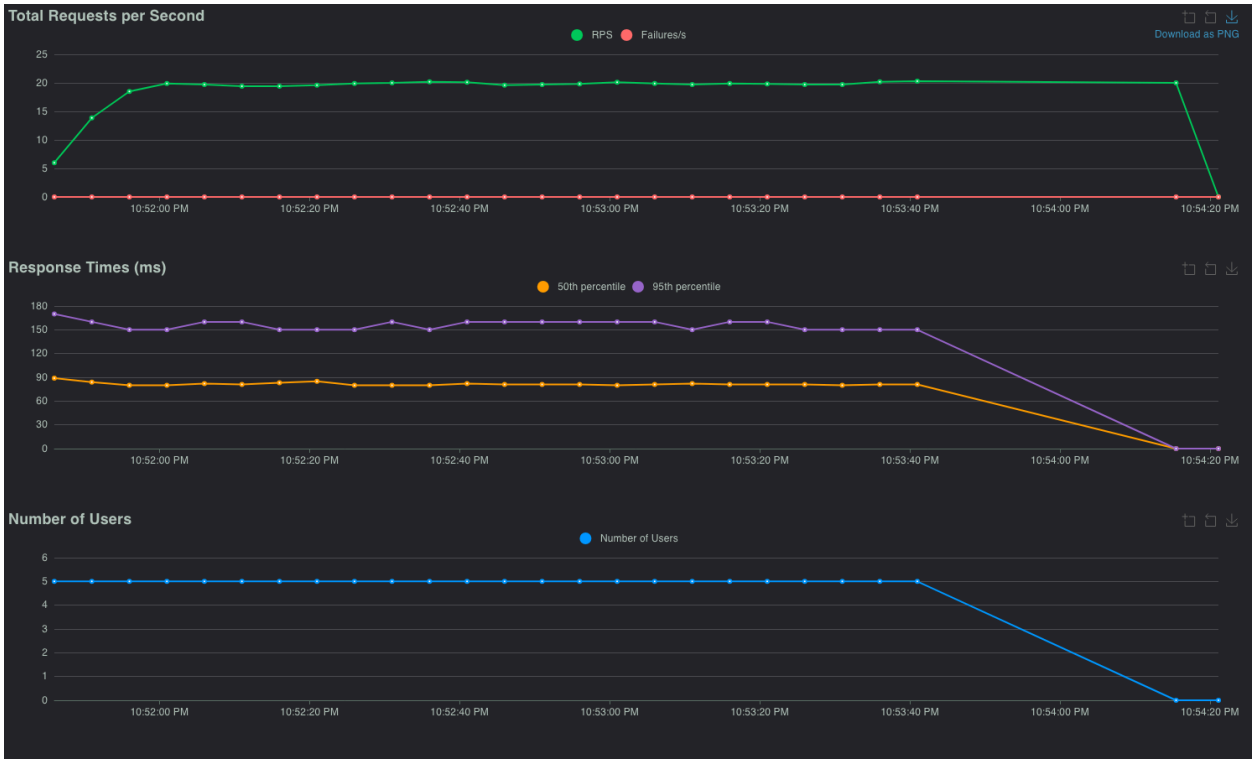
AWS:

Initial load tests with 5, 20, 100, 200, and even 500 concurrent users did not reveal a clear breaking point. The system demonstrated near-linear scaling in throughput, with stable response times and 0% failures across all runs. Even under high concurrency, the service sustained the offered load without observable degradation or crashes.

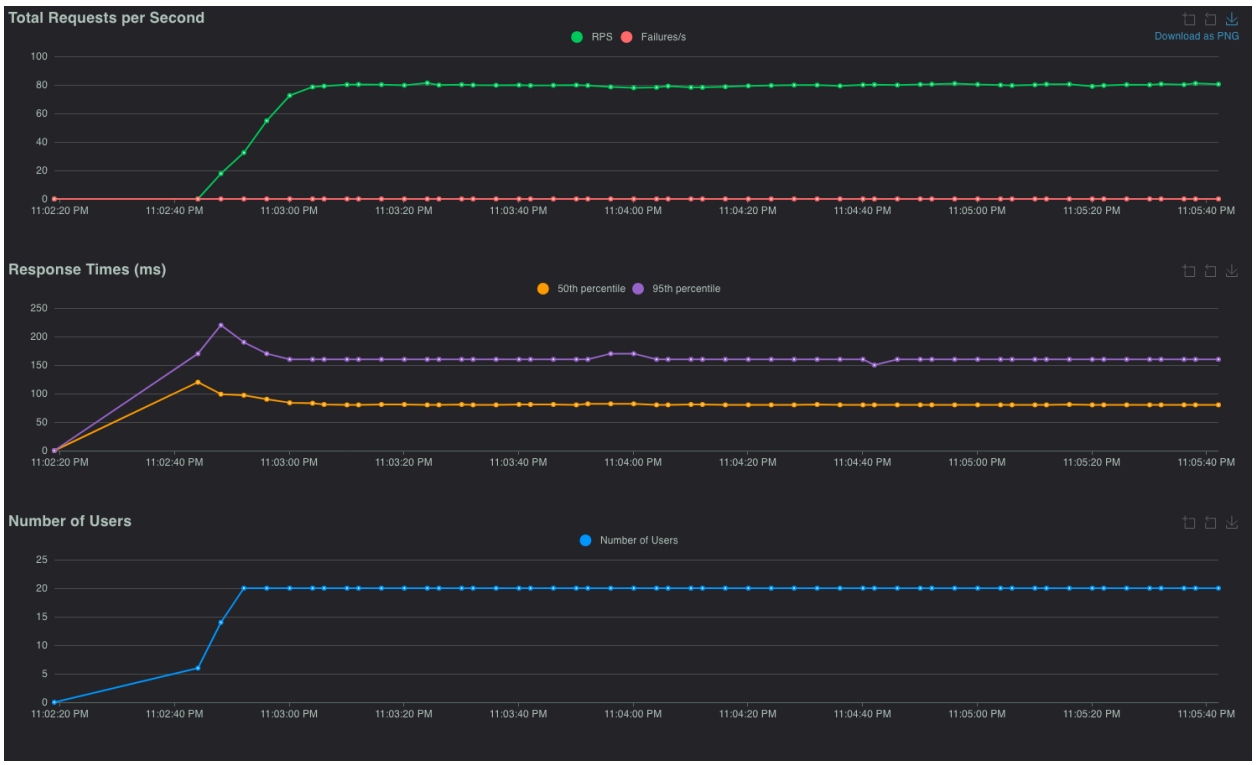
To further stress the system, the maximum number of products checked per request was increased to 10,000, significantly raising the CPU cost per search. Under this configuration, the ECS task's CPU utilization reached approximately 100%, indicating resource saturation. At this stage, throughput plateaued and response times increased noticeably, demonstrating that the service had become CPU-bound.

However, even at full CPU utilization, the service did not fail or crash. Instead, it continued processing requests successfully, albeit with higher latency. Therefore, the breaking point in this system was characterized by CPU saturation and increased response times rather than request failures or system instability.

1. Test 1 - Baseline: 5 users for 2 minutes

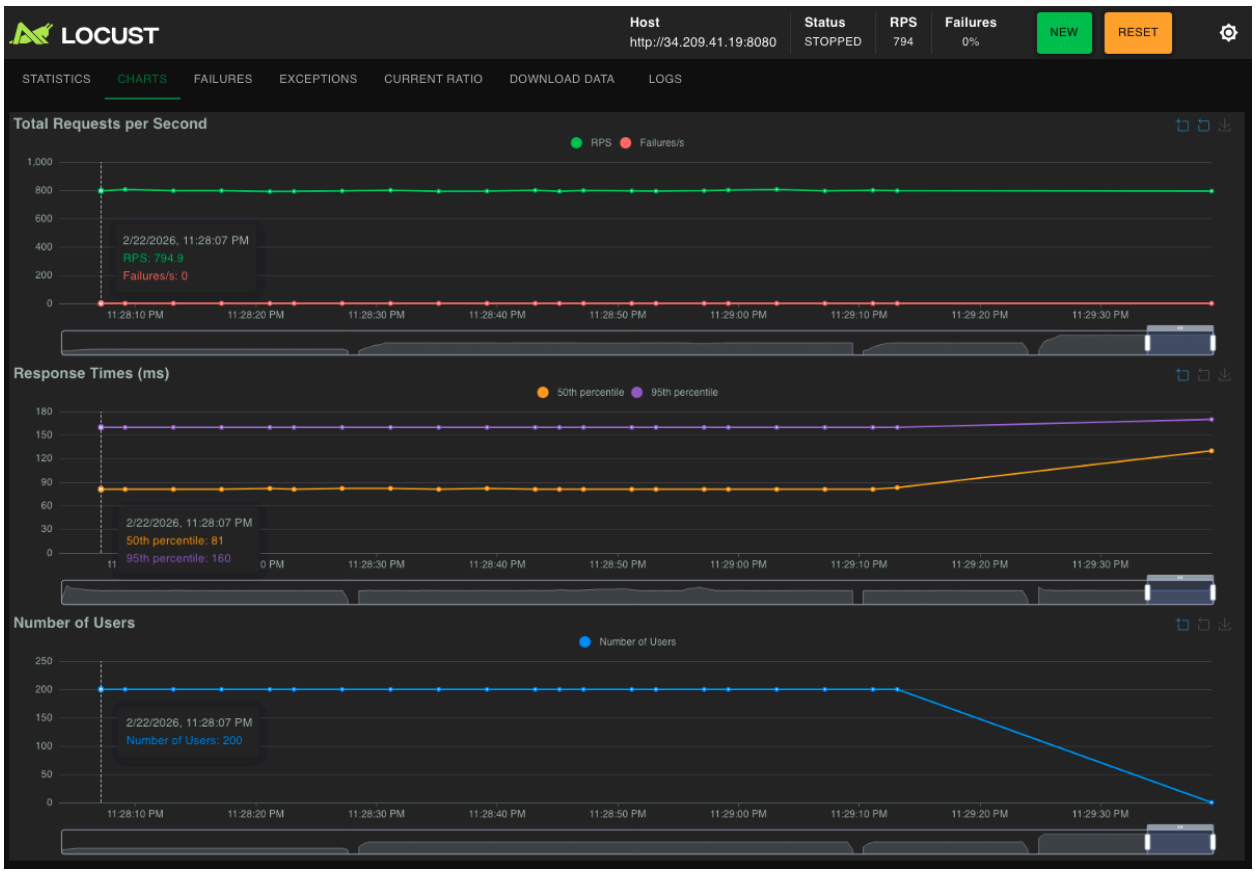


2. Test 2 - 20 users for 3 minutes (the server continued to operate - not a breaking point)



3. Test 3 - 100 users for 5 minutes (the server continued to operate - not a breaking point)

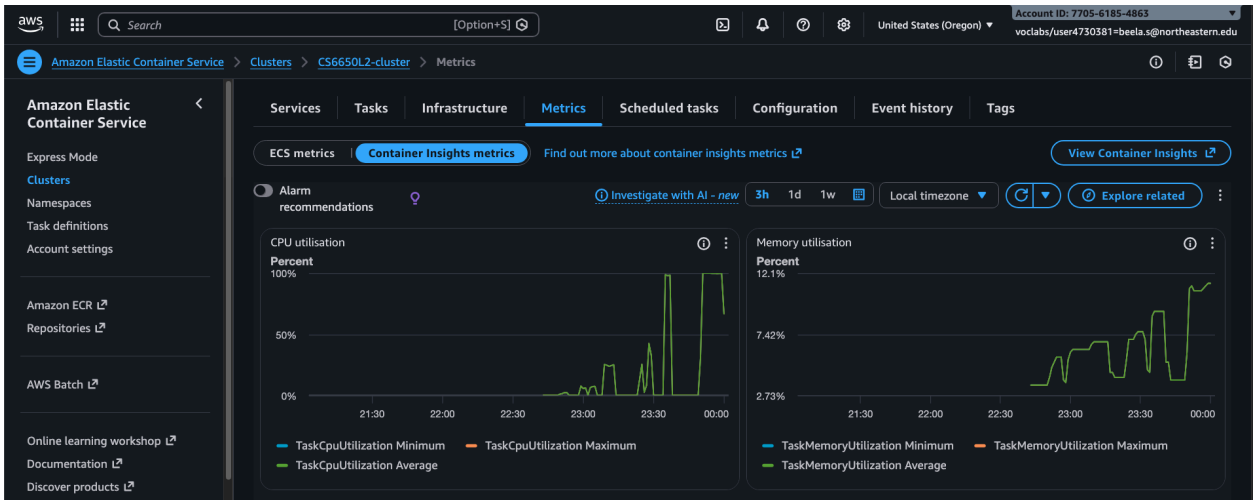
4. Test 4 - 200 users for 3 minutes (the server continued to operate - not a breaking point)



5. Test 4 - 500 users for 4 minutes (the server continued to operate - not a breaking point)

6. Test 5 - Breaking Point: 800 users for 10 minutes (the server continued to operate at 99% CPU Utilisation)

CPU Utilisation Metrics



Q/As

Q) Which resource hits the limit first?
A) CPU. Memory stayed mostly flat while CPU rose toward saturation under load.

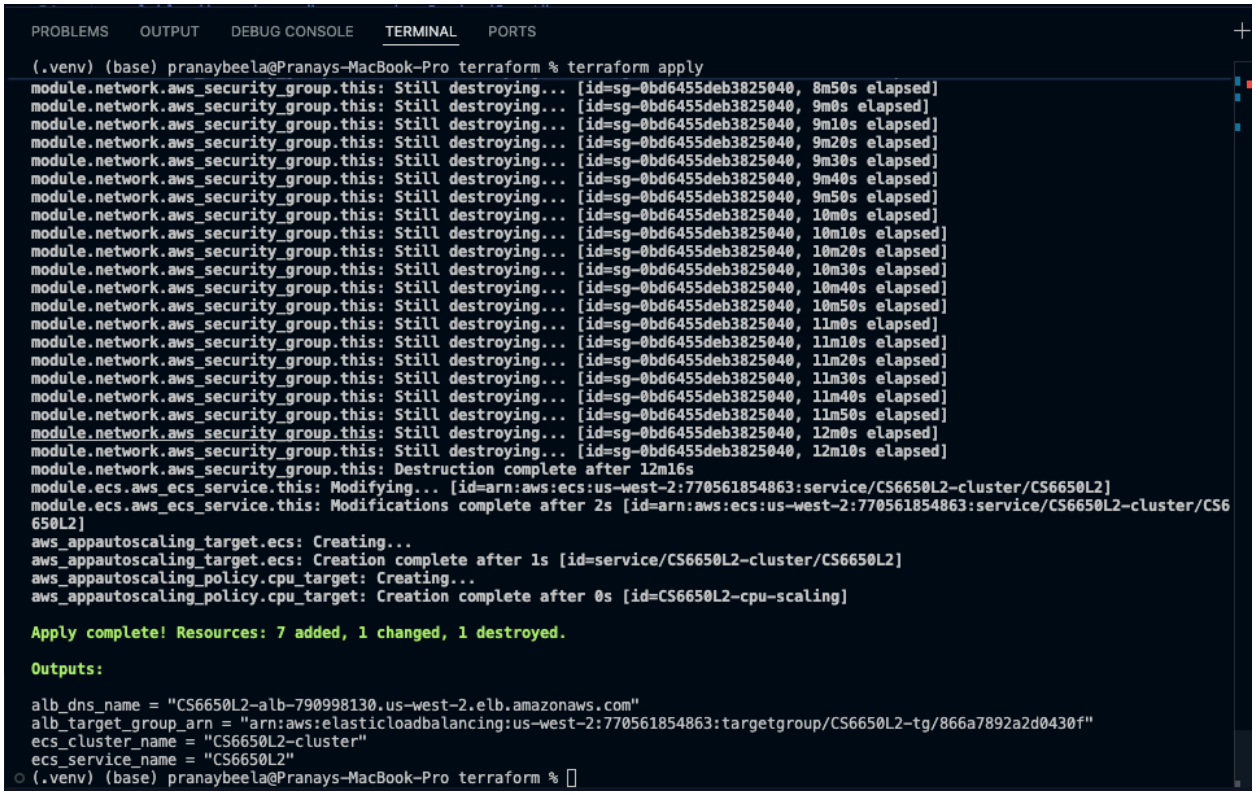
Q) How much did response times degrade?
A) As load increased, p95 latency rose noticeably and throughput began to plateau. At high load, p90 ~190–200 ms and p95 ~280–300 ms, showing a clear latency jump compared to baseline.

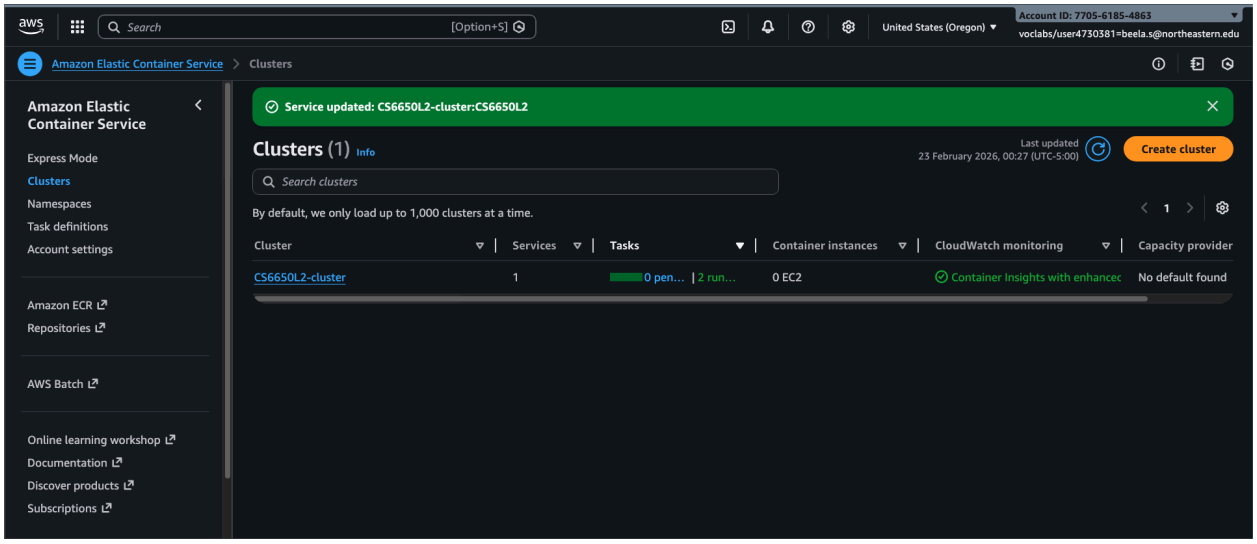
Q) Could you solve this by doubling the CPU (256 → 512 units)?
A) Yes, likely. The workload is fixed-cost per request, so more CPU should raise throughput and reduce queueing, without needing code changes.

Part III - Horizontal Scaling with Auto Scaling

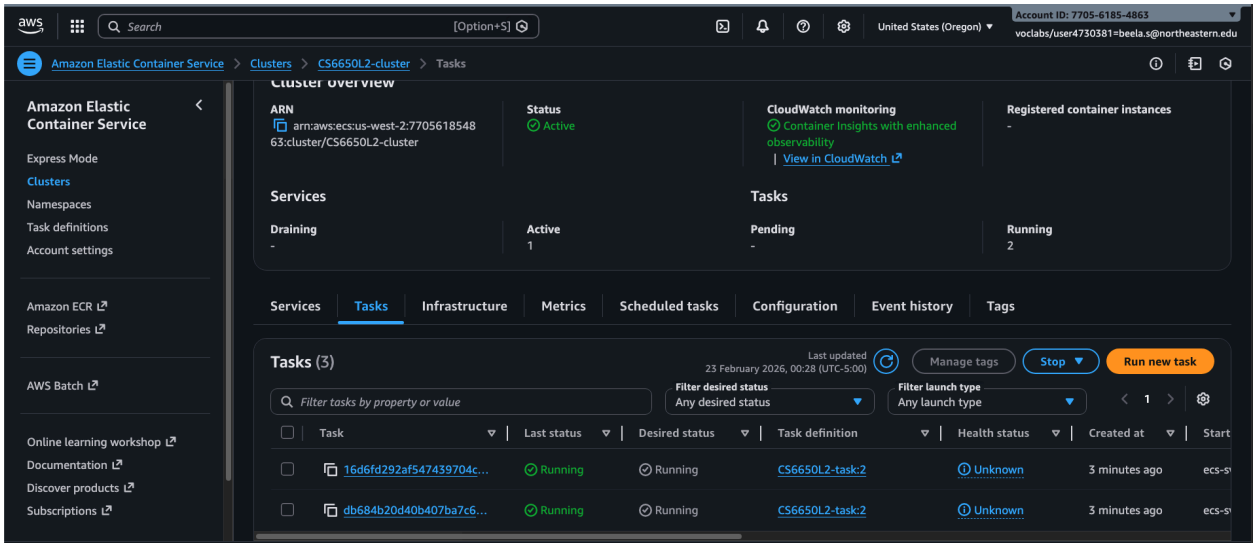
Setup:

The Application Load Balancer (ALB) was successfully provisioned. Terraform outputs a public DNS name, which is used as the base URL for load testing.



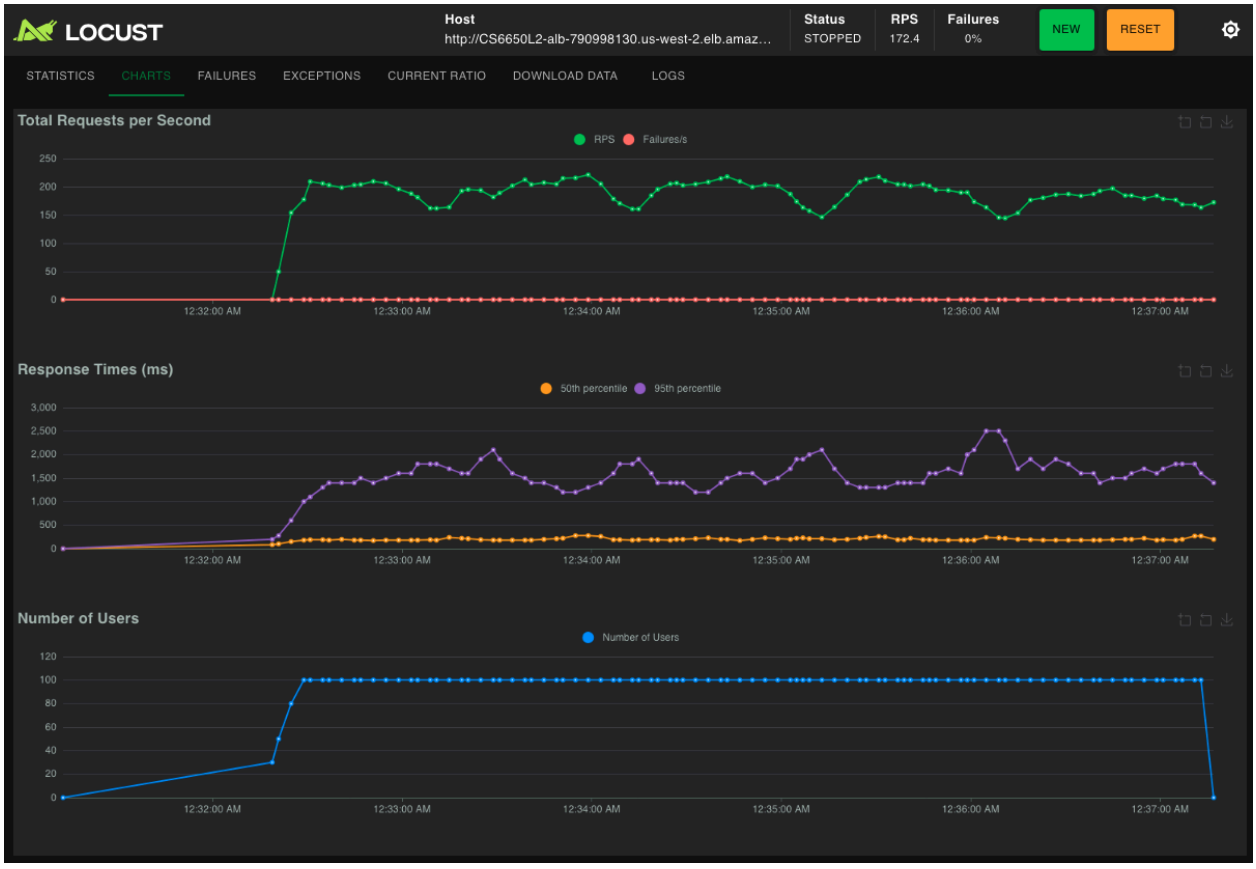


I initialised with 2 tasks (minimum) at first.



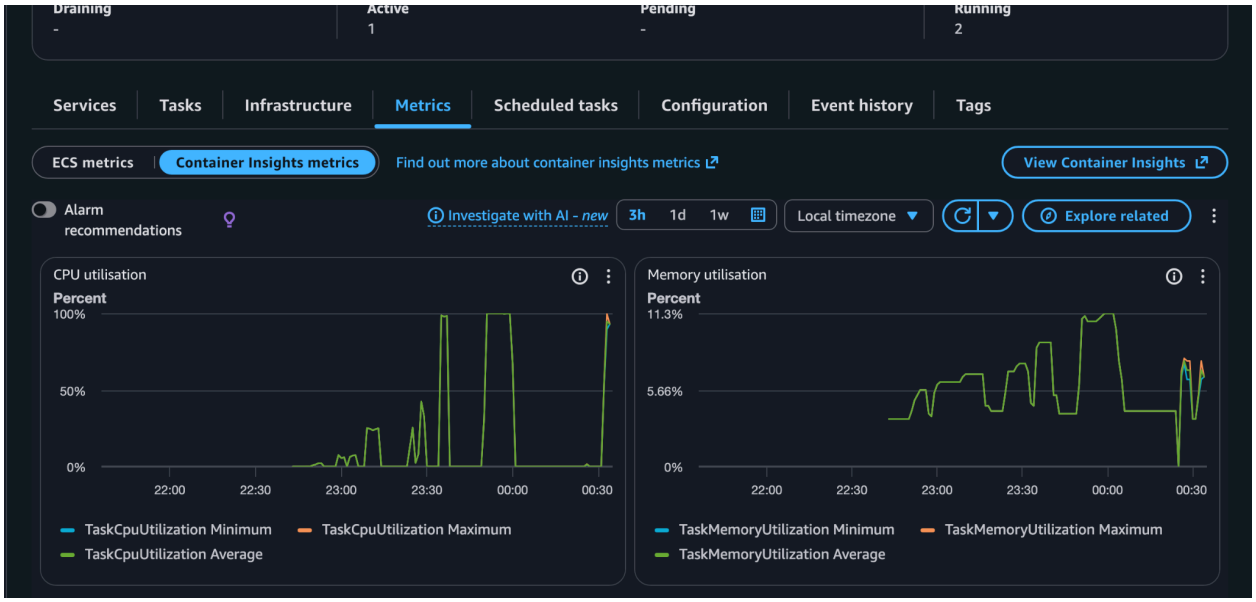
Load Test Results:

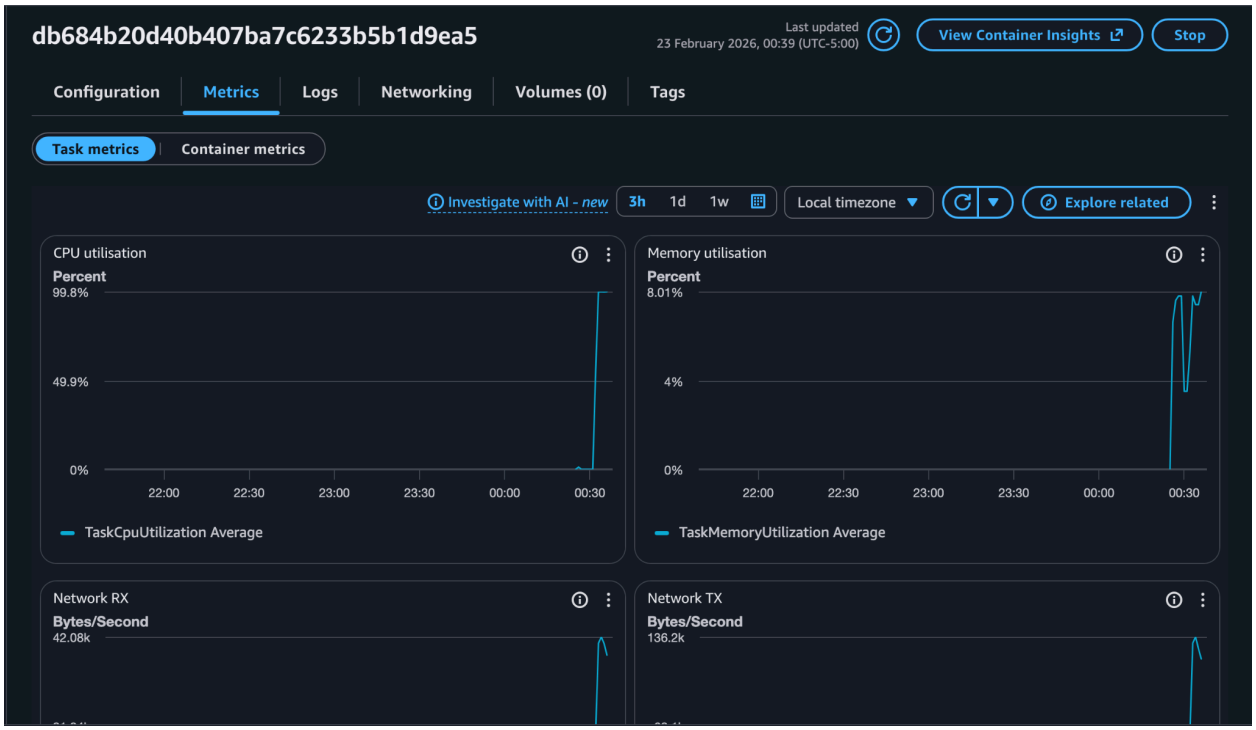
Test - 100 users for 5 minutes

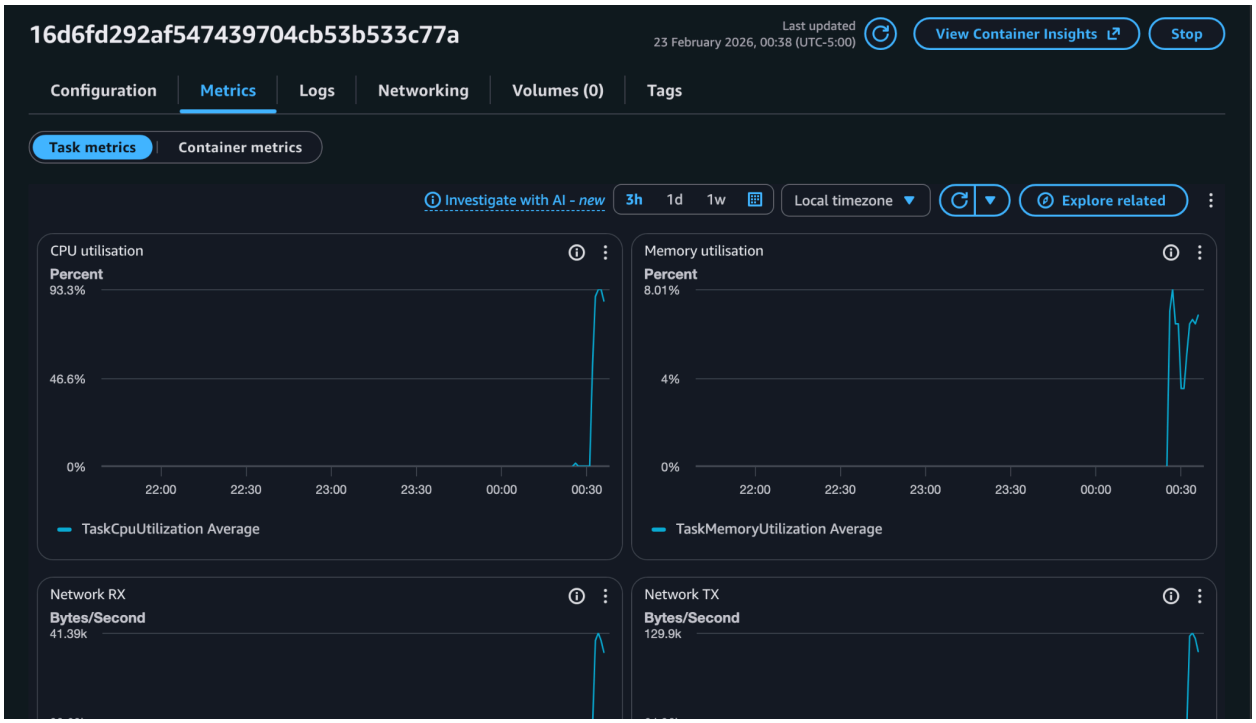


The instance handled the above test well. During the 100-user, 5-minute run, autoscaling did not trigger, even though CPU utilization peaked. The service remained stable with no failures. Since the deployment started with two tasks, the load was distributed across both, which helped maintain performance. But, as we can see the response times remained pretty high (1500ms-2000ms) most of the time.

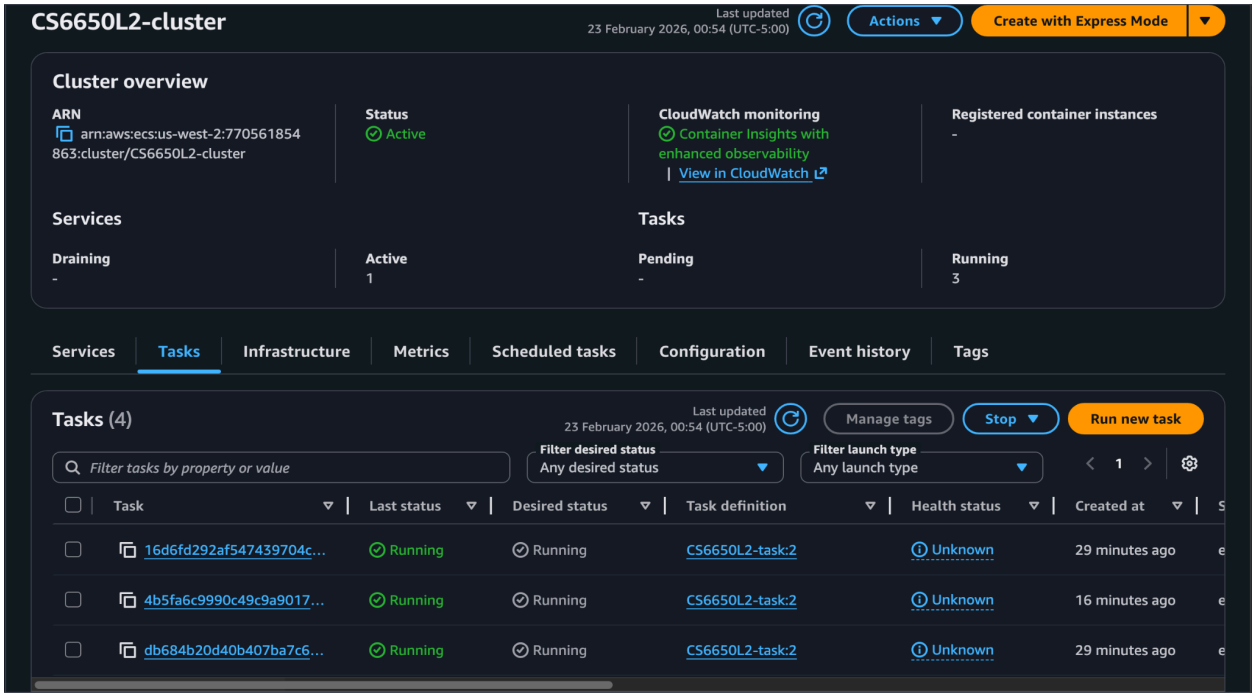
CPU Utilisations - Combined & Individual





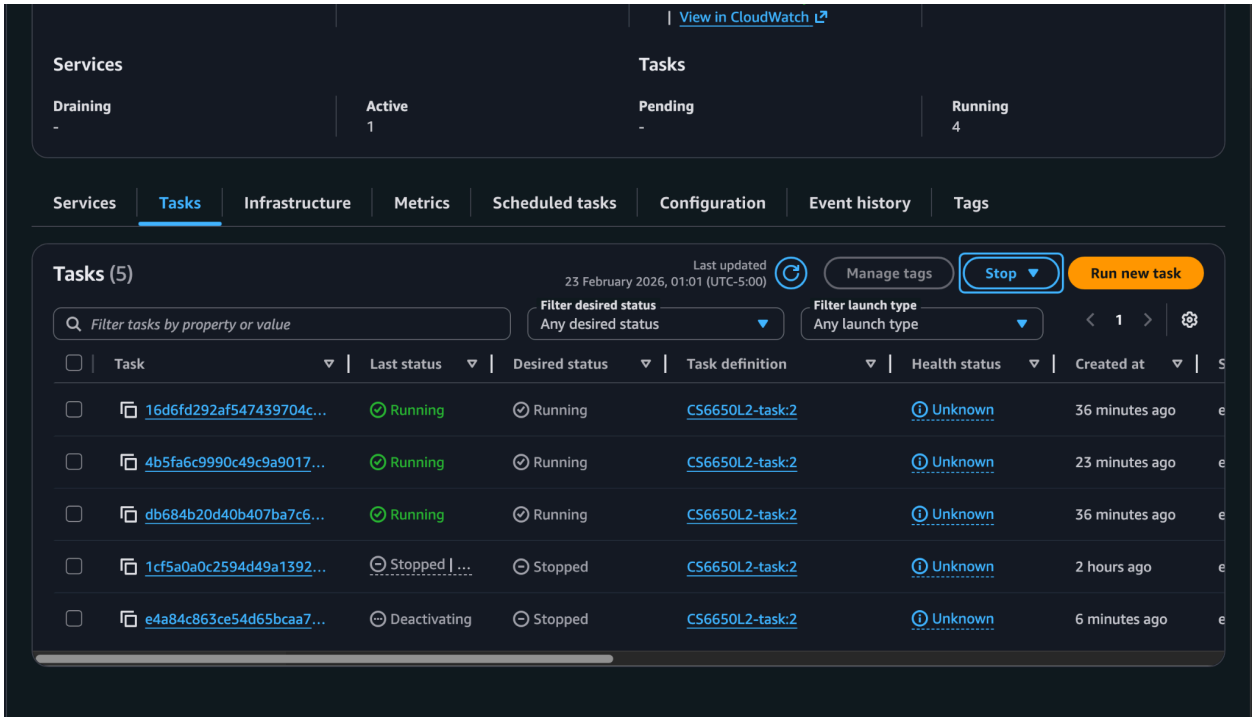


I reran the same test for 15 minutes with the ALB/ECS CPU target set to 70%. This means the service should scale out if CPU utilization remains around or above 70% for a sustained period. I also configured a 300-second cooldown for both scale-out and scale-in, so the system waits 300 seconds before adding a new task or reducing the task count based on observed CPU utilization.

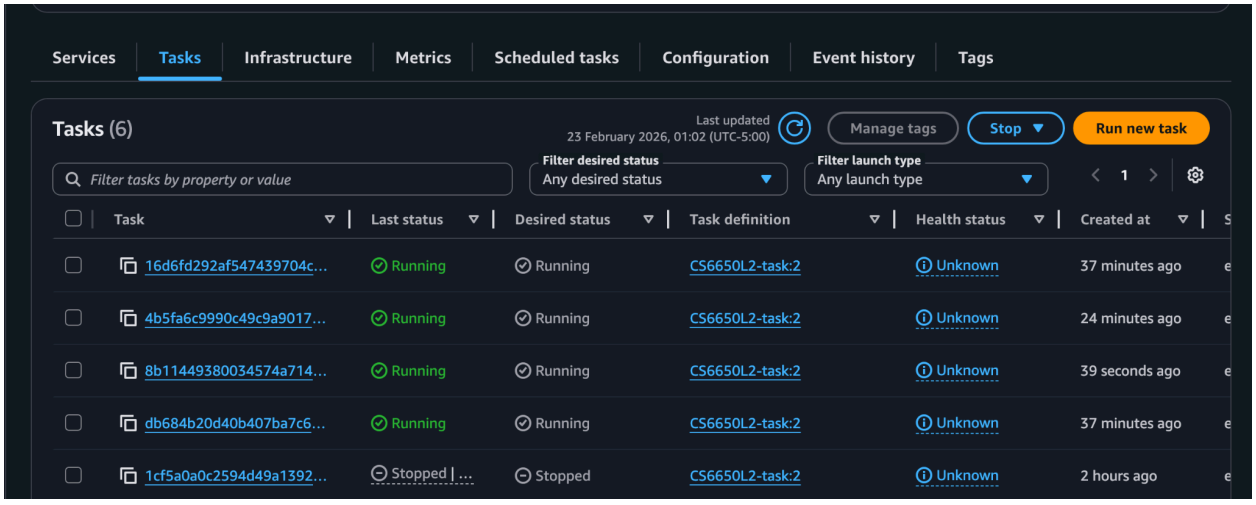


As shown in the image, a new ECS task was automatically launched once CPU utilization remained at or above the 70% target for a sustained period. This confirms that the ALB target-tracking scaling policy was triggered successfully and scaled out the service as expected.

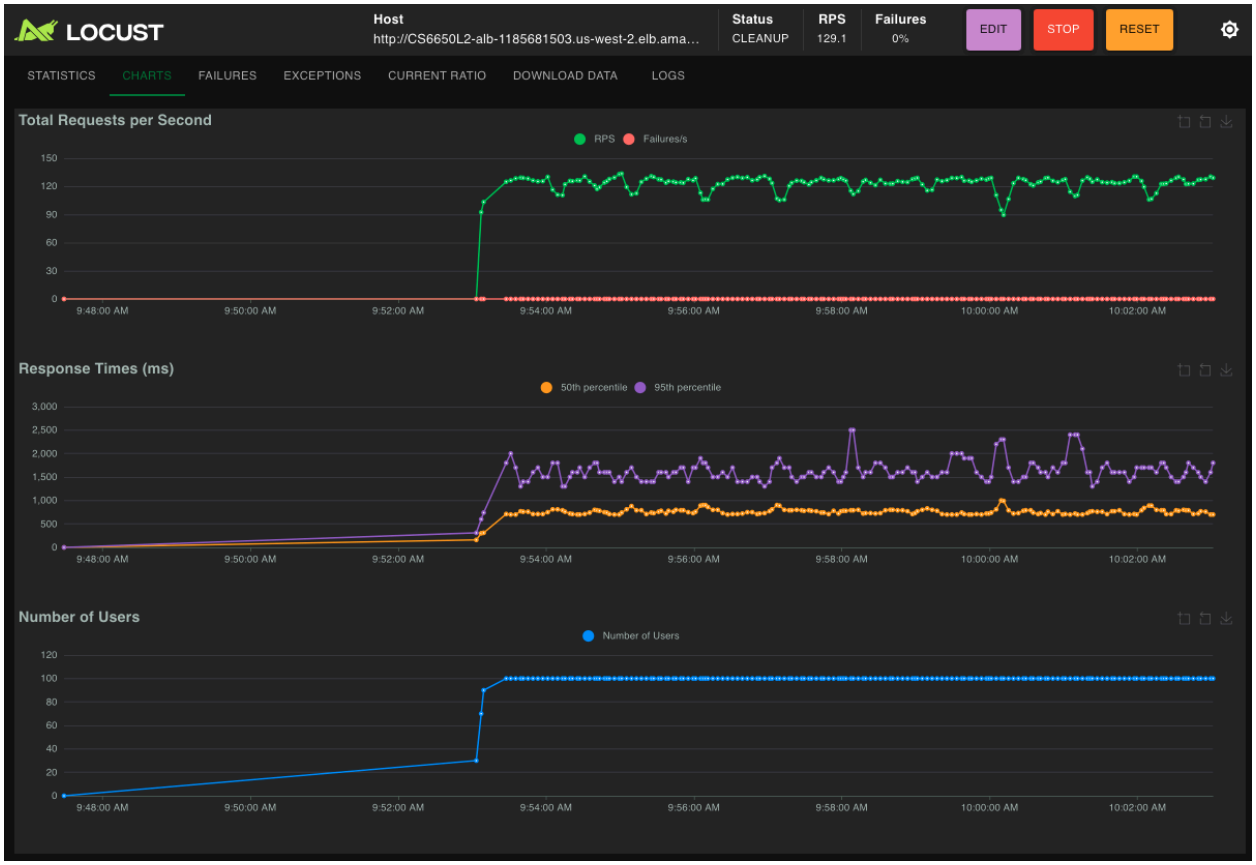
I abruptly stopped one of the running tasks, and the service continued operating, but the p95 response time increased due to reduced capacity. After the 300-second cooldown period, ECS automatically launched a new task to restore capacity and handle the load.



Automatic task provision:

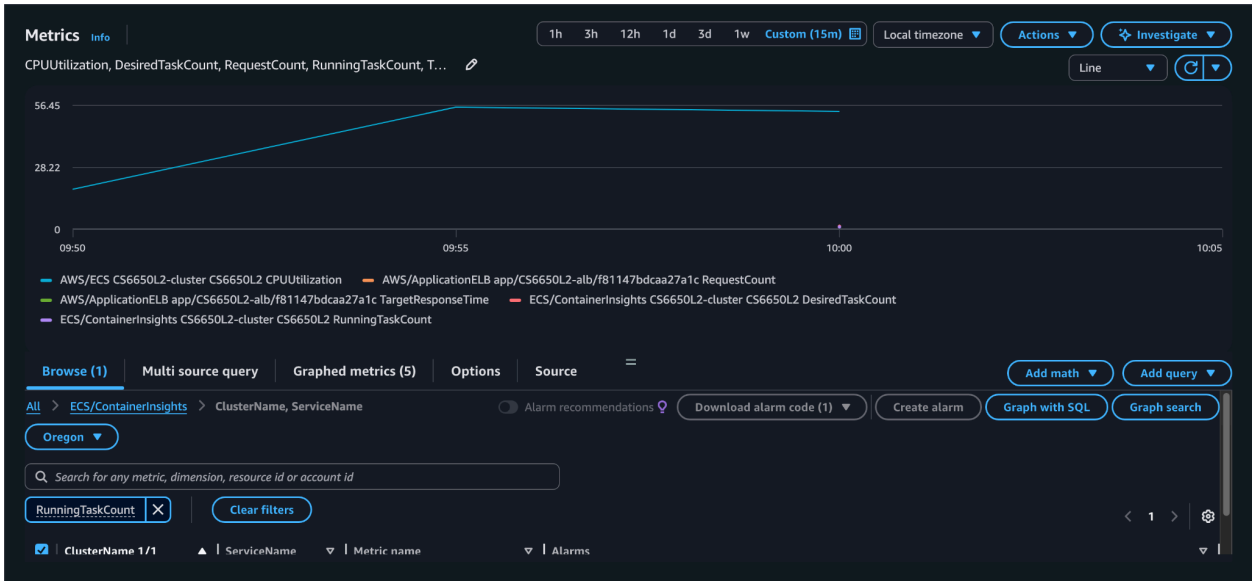


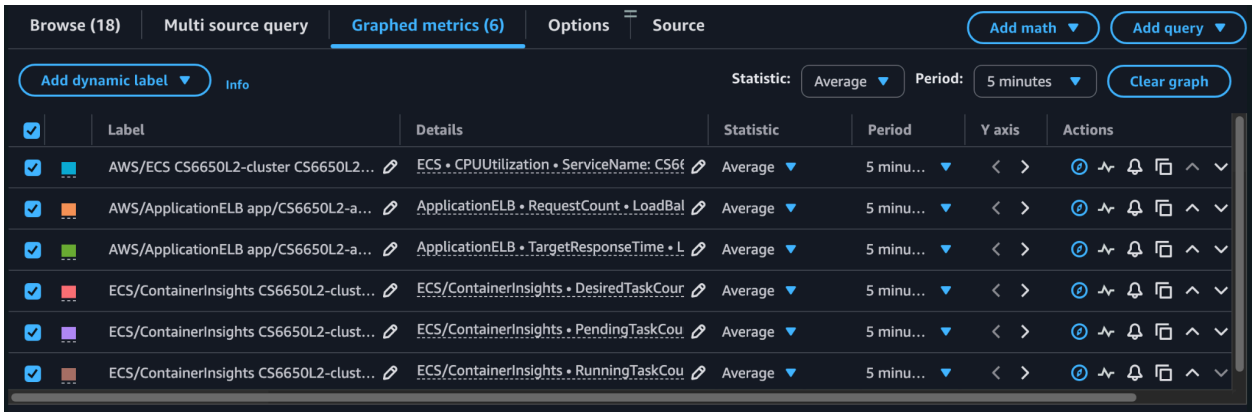
Now, I tried changing the auto scaling default values. I set the min to 1 and max to 6 and also the CPU target at which scaling should happen to 90%.



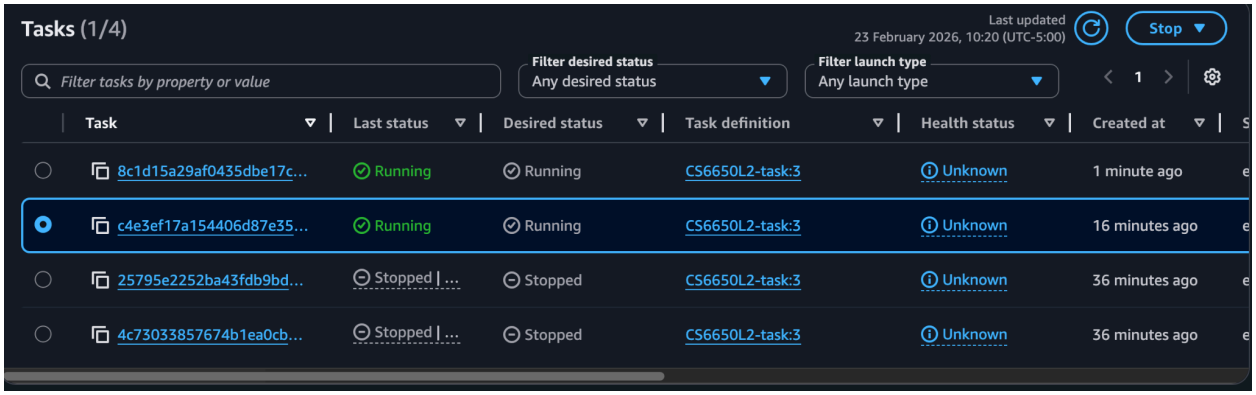
During this test, the system was subjected to sustained load with 100 concurrent users through the ALB endpoint. Throughput stabilized at approximately 120–130 RPS with no request failures observed. However, response times increased significantly compared to earlier tests, with p95 latency reaching around 1.5–2.5 seconds, as there was only one task running but was still able to handle the load.

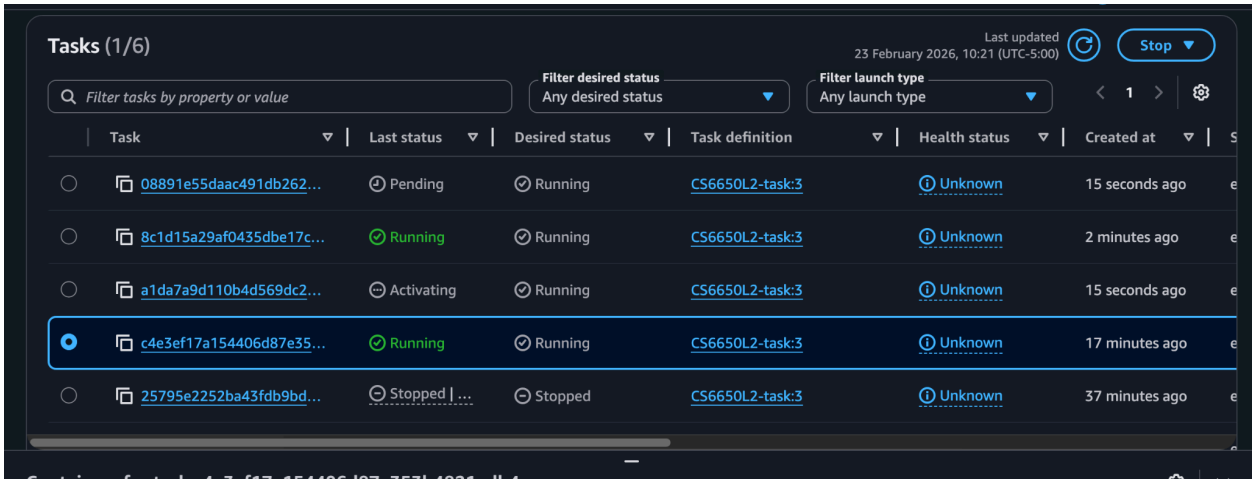
Cloudwatch Metrics at 90% CPU Utilization with one ECS Task





I adjusted the autoscaling configuration by setting the minimum tasks to 1 and the maximum to 6, lowering the CPU target to 50%, and reducing both scale-in and scale-out cooldowns to 60 seconds. With this configuration, ECS began launching additional tasks as soon as CPU utilization sustained around the 50% threshold, resulting in faster scale-out behavior under load.



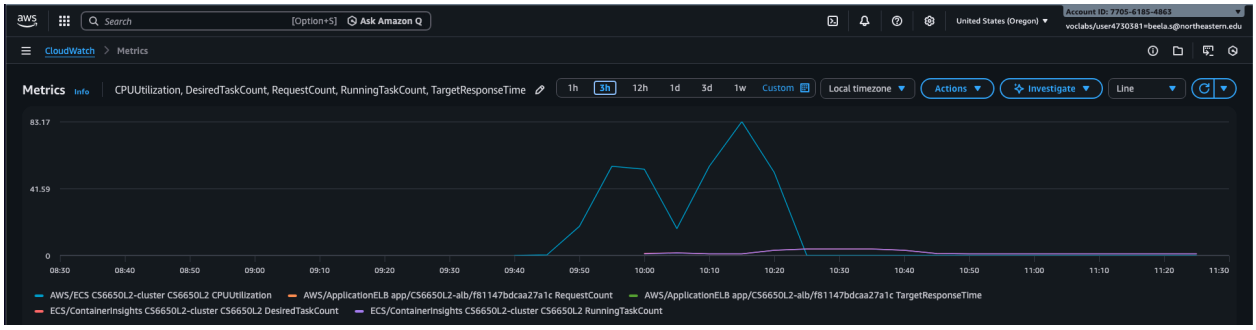


As shown in the image, when only one task was running, the system experienced high latency, with p95 response times ranging between 6000–8000 ms, and throughput was capped at approximately 150 RPS. This indicates that the service was CPU-bound under the given load.

Once CPU utilization crossed the configured 50% threshold, the autoscaling policy triggered and additional ECS tasks were launched. Following scale-out, throughput increased significantly to approximately 600 RPS, while p95 latency dropped to around 2000 ms.

This demonstrates the effectiveness of the target-tracking autoscaling configuration, where distributing the load across multiple tasks reduced per-task contention and substantially improved response times.

The cluster performance view shows that CPU utilization briefly spiked close to 100% during load testing before dropping back to near idle levels once the load subsided. Memory utilization remained relatively low (around 5–10%), indicating that the workload was primarily CPU-bound rather than memory-bound. The running task count reflects the scaling behavior observed earlier, where additional tasks were provisioned to handle increased demand.



The CloudWatch dashboard provides a consolidated view of CPUUtilization, DesiredTaskCount, RunningTaskCount, RequestCount, and TargetResponseTime. The CPU spikes align with increases in request traffic, after which autoscaling adjusted the running task count accordingly. As additional tasks were launched, CPU utilization stabilized and target response times improved. This confirms that the target-tracking autoscaling policy functioned as expected under sustained load.

Q/As

Discovery Questions

Q) How does the system respond to the load that broke Part A?
Response to Part 2 breaking load: The system stayed available and throughput remained steadier because new tasks were added under load.

Q) When do new instances get added?
When new instances get added: After average CPU stays above the target (e.g., 50%/70%) for the cooldown window (~60s in your config).

Q) How is the load distributed across instances?
Load distribution: The ALB spreads requests across all healthy targets; as tasks scale out, each task handles a smaller share.

Q) What happens to response times as instances scale?
Response times as instances scale: p95 latency stabilizes or drops once additional tasks come online.

Results

1. How the system solved the Part 2 bottleneck
A) In Part 2, the application was CPU-bound because it ran on a single ECS task, limiting total compute capacity. In Part 3, introducing an ALB distributes incoming traffic across multiple tasks, while target-tracking auto scaling provisions additional tasks when average CPU utilization exceeds the configured threshold. This increases total available CPU capacity, reduces per-task load, and maintains lower p95 latency under the same traffic levels.

2. Role of each component
A) ALB (Application Load Balancer): Acts as the public entry point and distributes incoming HTTP requests across healthy ECS tasks.

Target Group: Monitors task health via health checks and provides the ALB with the set of healthy routing targets.

Auto Scaling (ECS Service Auto Scaling): Dynamically adjusts the number of running ECS tasks based on CPU utilization to match workload demand.

3. Trade‑offs vs vertical scaling
A) Horizontal scaling improves resilience, fault tolerance, and elasticity by distributing traffic across multiple tasks. However, it introduces additional architectural complexity (ALB configuration, health checks, scaling policies, and monitoring).

Vertical scaling is simpler to configure but is constrained by the limits of a single instance and remains a single point of failure. It also does not provide the same elasticity under fluctuating load.

4. Predicted scaling behavior

A) For sudden traffic spikes, scaling reacts only after CPU metrics breach the configured threshold and cooldown conditions are satisfied. This introduces a short lag before newly launched tasks begin handling traffic, during which p95 latency may temporarily increase.

For sustained high load, tasks scale out progressively until reaching the configured maximum capacity, stabilizing average CPU utilization near the target threshold.

For gradual load increases, scaling behavior is smoother, with fewer latency spikes, as additional tasks are provisioned incrementally in response to steady metric changes.

5. Creative stress testing evidence
A) I executed progressive load tests at multiple levels (5 → 20 → 100 → 200 → 500 → 800 users) to observe system behavior under increasing traffic. In addition, I deliberately terminated running tasks during active load to evaluate ALB target group health checks and ECS task replacement behavior.

I also experimented with autoscaling configurations by modifying minimum and maximum task counts, adjusting CPU utilization target thresholds (e.g., 70% → 50%), and tuning cooldown periods. This allowed me to analyze how different scaling policies impacted throughput (RPS), p95 latency, and overall cluster stability under sustained stress.